Intro to Deep Learning

Lecture 01: Neural Networks, MLPs, and Activations

Stanislav Don

DS212 - Intro to Deep Learning

Course Highlights

This course is not a catalogue of model names.

It is a path from **classical ML intuition** to **deep learning engineering**.

01

Training kitchen

tensors, autograd, losses, optimizers, custom PyTorch loops

02

Computer vision

CNNs, augmentation, ResNet, transfer learning, YOLO

03

Transformers & LLMs

attention, tokenization, fine-tuning, SFT, LoRA, RLHF ideas

04

Multimodal ML

embeddings, retrieval, CLIP, image-text representation spaces

Goal: understand models well enough to debug, adapt, and build with them.

Course Timing

Module dates:

May 18, 2026 - June 5, 2026

Each lesson is **3 hours**.

First 1.5 hours

Theory, concept analysis, visuals, math anchors, and live coding.

Second 1.5 hours

Seminar practice, PyTorch tasks, Kaggle contests, and project work.

Grading And Deliverables

Contests
40%
2 parts out of 5

Homeworks
40%
2 parts out of 5

Participation
20%
1 part out of 5

Contests are graded by more than leaderboard position:

- private leaderboard score
- reproducible notebook or code
- model comparison / ablation
- error analysis

Participation means active seminar work:

- asking and answering questions
- coding during practice
- debugging and discussing results

Homework And Contest Timeline

Item	Release	Deadline	Duration
HW HW0: derivatives	May 18	May 21	3 days
HW HW1: overfitting / regularization	May 21	May 24	3 days
Contest Food-20	May 22	May 28	~1 week
HW HW2: visual search / embeddings	May 28	May 31	3 days
Contest Banking77	May 29	June 5	1 week
Optional YOLO	May 29	June 5	1 week

Plan ahead: contests overlap with lectures, so start early and iterate.

Why These Deliverables Exist

HW0

Refresh derivatives and gradients before backprop.

HW1

Practice validation, overfitting diagnosis, and regularization.

Food-20

Build an image classifier; compare CNN and transfer-learning choices.

HW2

Use embeddings for visual search and retrieval.

Banking77

Fine-tune and evaluate text classifiers for many intent classes.

Optional YOLO

Extra object-detection practice for students who want more CV.

The goal is not only a score: the goal is reproducible experiments and error analysis.

Today

By the end of the lecture, you should be able to answer:

1. What is a neural network?
2. What does a linear layer compute?
3. Why do we need nonlinear activations?
4. What happens during a forward pass?
5. Why do tensor shapes matter so much?

Lecture rhythm: concept -> visual -> math anchor -> PyTorch shape

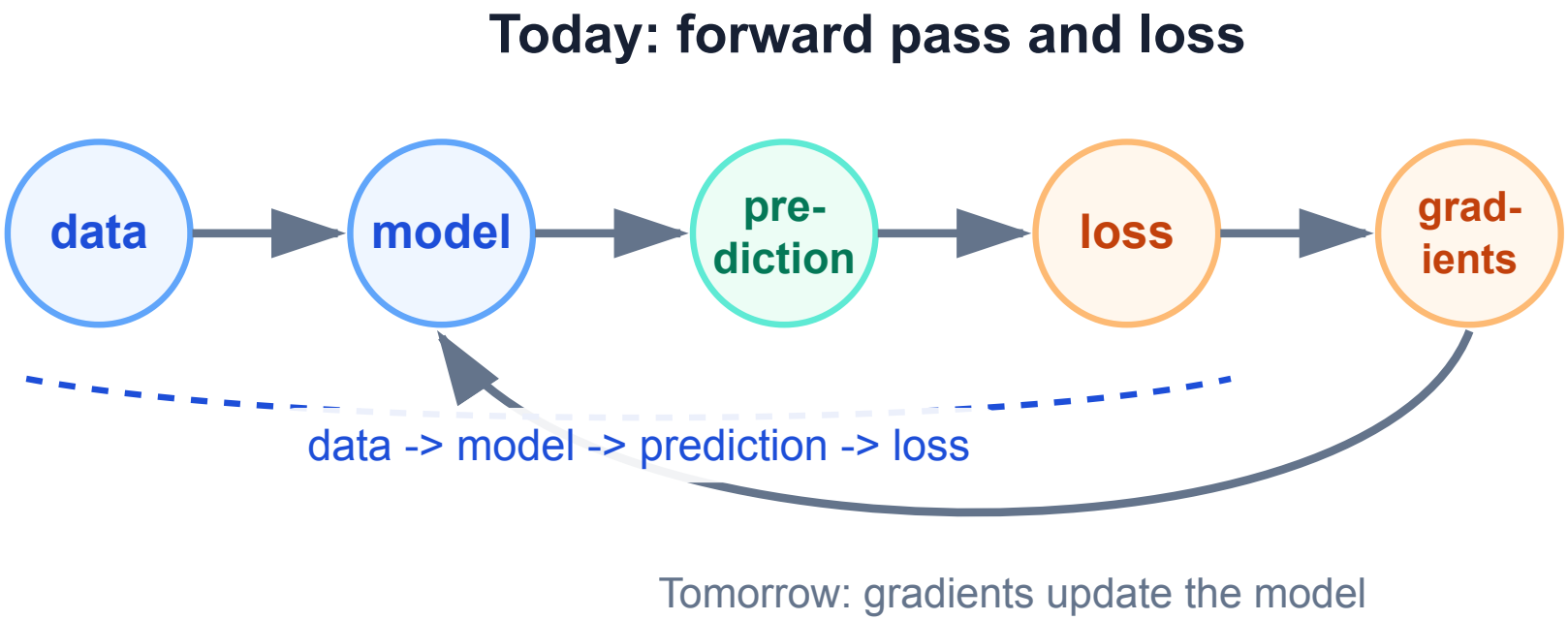
Part 1

The Course Mental Model

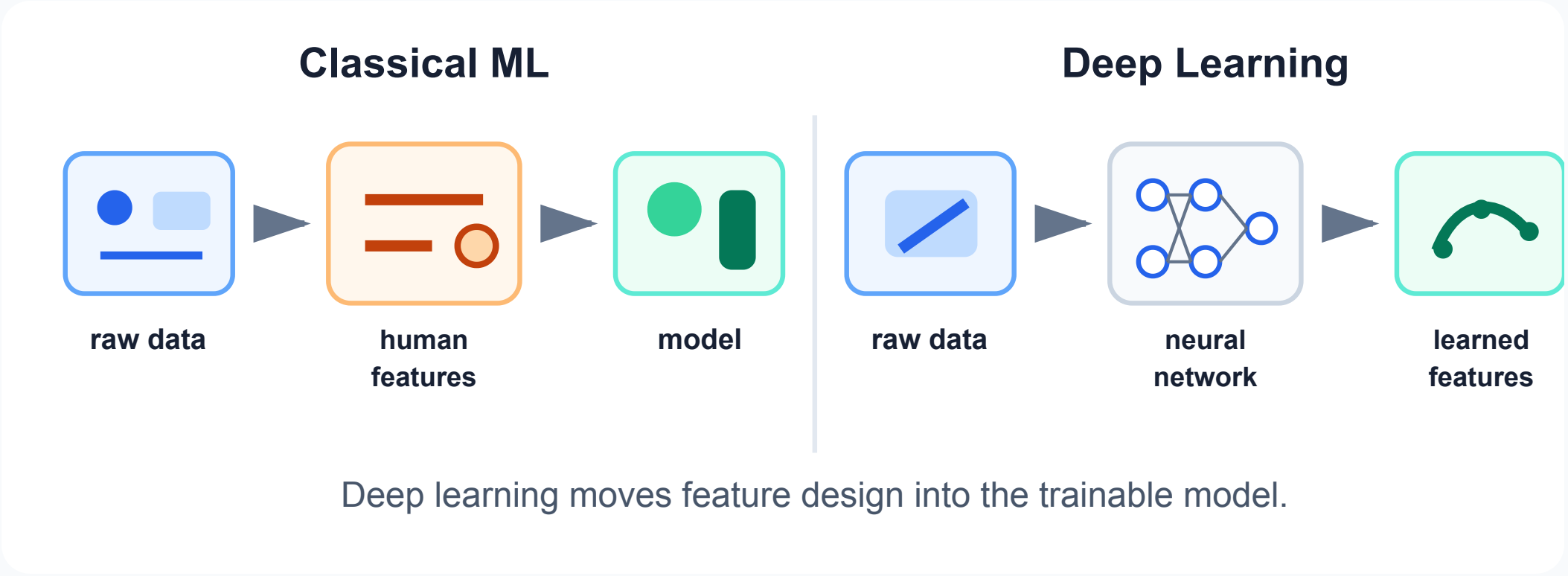
Deep learning as a repeated engineering loop

One Loop, Many Models

Almost every model in this course follows the same training pattern.



Classical ML vs Deep Learning

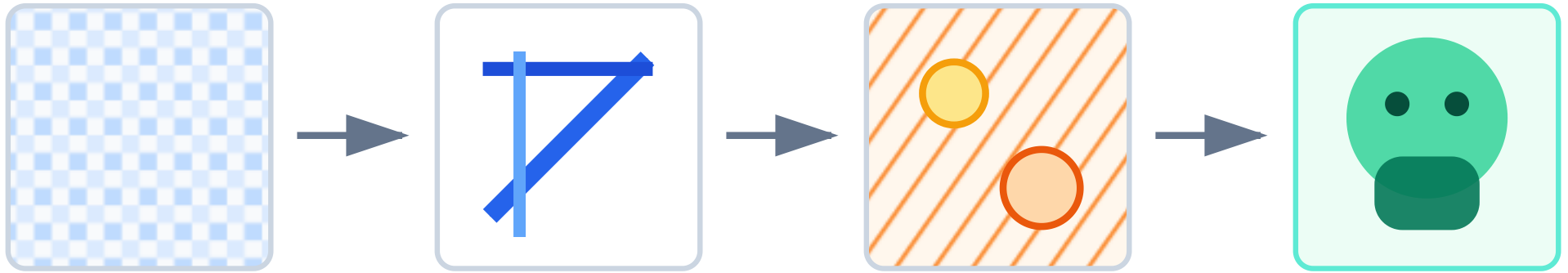


Deep learning moves feature design into the trainable model.

The price: more data, more compute, and more debugging discipline.

Learned Visual Representations

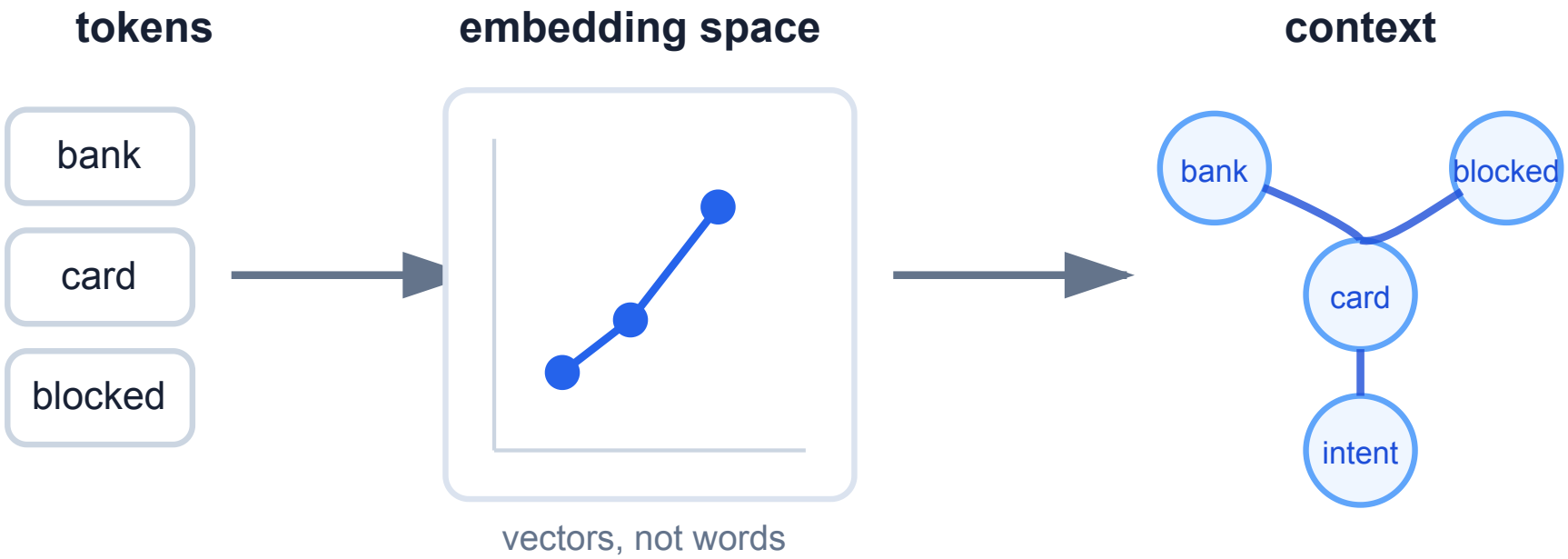
CNNs can learn a hierarchy of image features.



We will make this concrete in the CNN lectures.

Learned Text Representations

Transformers turn tokens into context-aware vectors.



We will make this concrete in the transformer lectures.

A Neural Network Is A Function

At the simplest level:

$$\hat{y} = f_\theta(x)$$

Where:

- x is the input
- $\hat{y}$ is the prediction
- θ are trainable parameters
- training means finding useful values of θ

The whole course is about designing, training, and debugging f_θ.

Part 2

Linear Building Blocks

We start with the simplest trainable transformation

Linear Model Recap

A linear model has the form:

$$\hat{y} = Wx + b$$

In PyTorch, this is `nn.Linear`.

The parameters are W and b.

Linear Layer Shapes

For one object:

```
x:     in_features
W:     out_features x in_features
b:     out_features
y_hat: out_features
```

For a batch:

```
X:     batch_size x in_features
Y_hat: batch_size x out_features
```

Shape discipline is not optional. Most DL bugs are shape bugs.

PyTorch Convention

In math, we often write:

$$\hat{y} = Wx + b$$

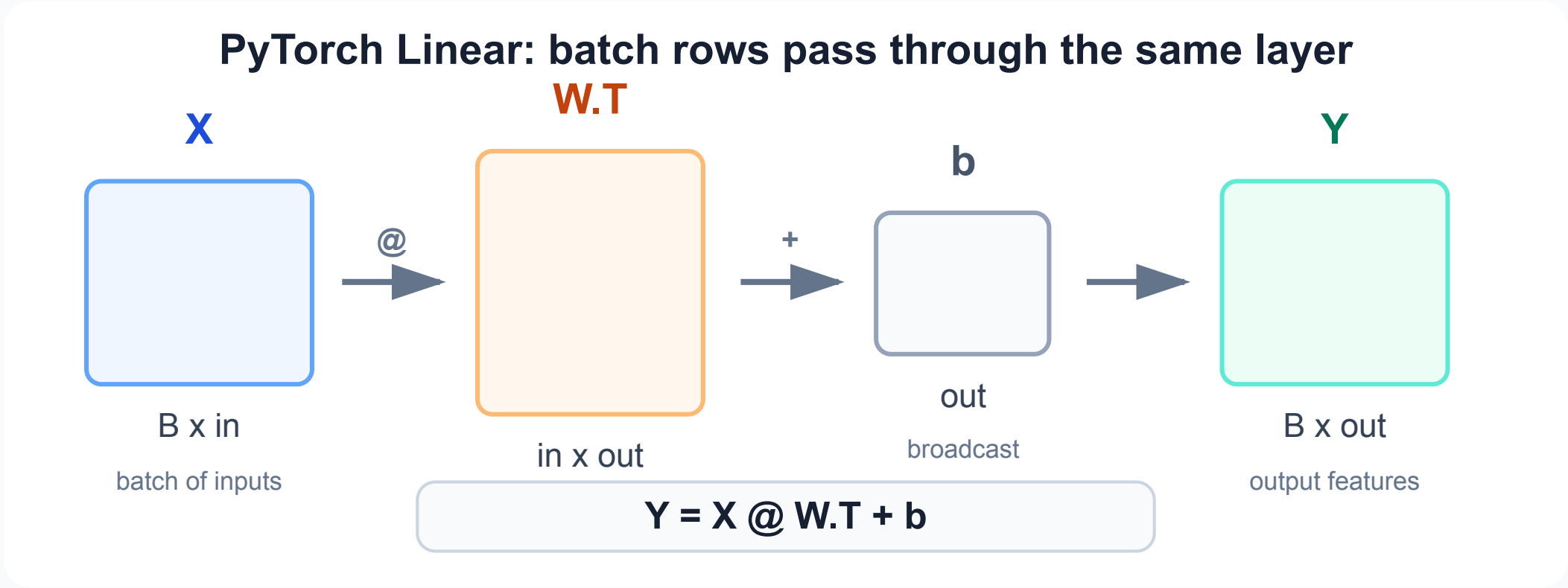
In PyTorch, batches are stored as rows:

```
Y = X @ W.T + b
```

For `nn.Linear(in_features, out_features)`:

```
weight shape: out_features x in_features
bias shape:   out_features
```

Visual Shape Check



Parameter Count

For a linear layer:

```
Linear(in_features, out_features)
```

Number of parameters:

$$in\_features \times out\_features + out\_features$$

Example:

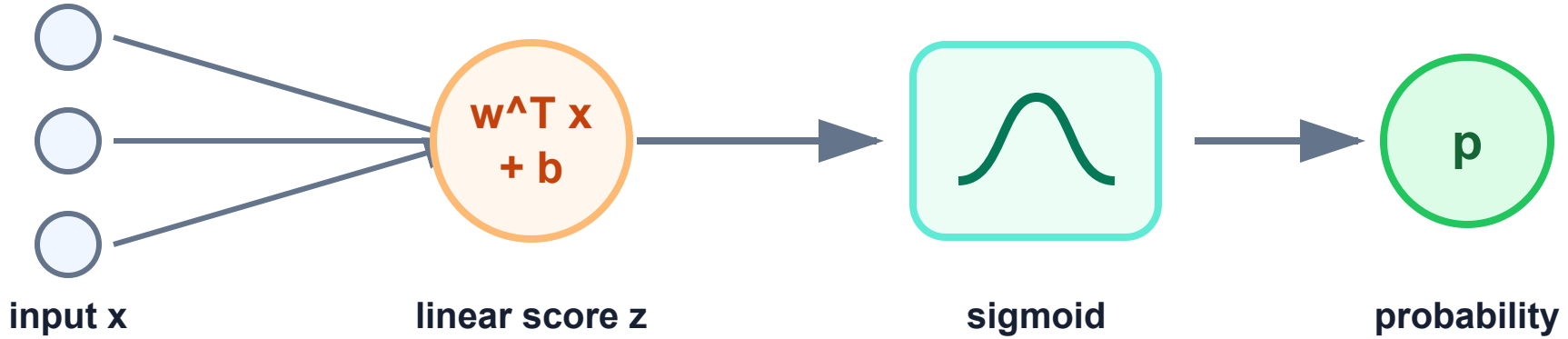
```
Linear(784, 10)
parameters = 784 * 10 + 10 = 7850
```

The bias contributes one parameter per output feature.

Logistic Regression As A Tiny Network

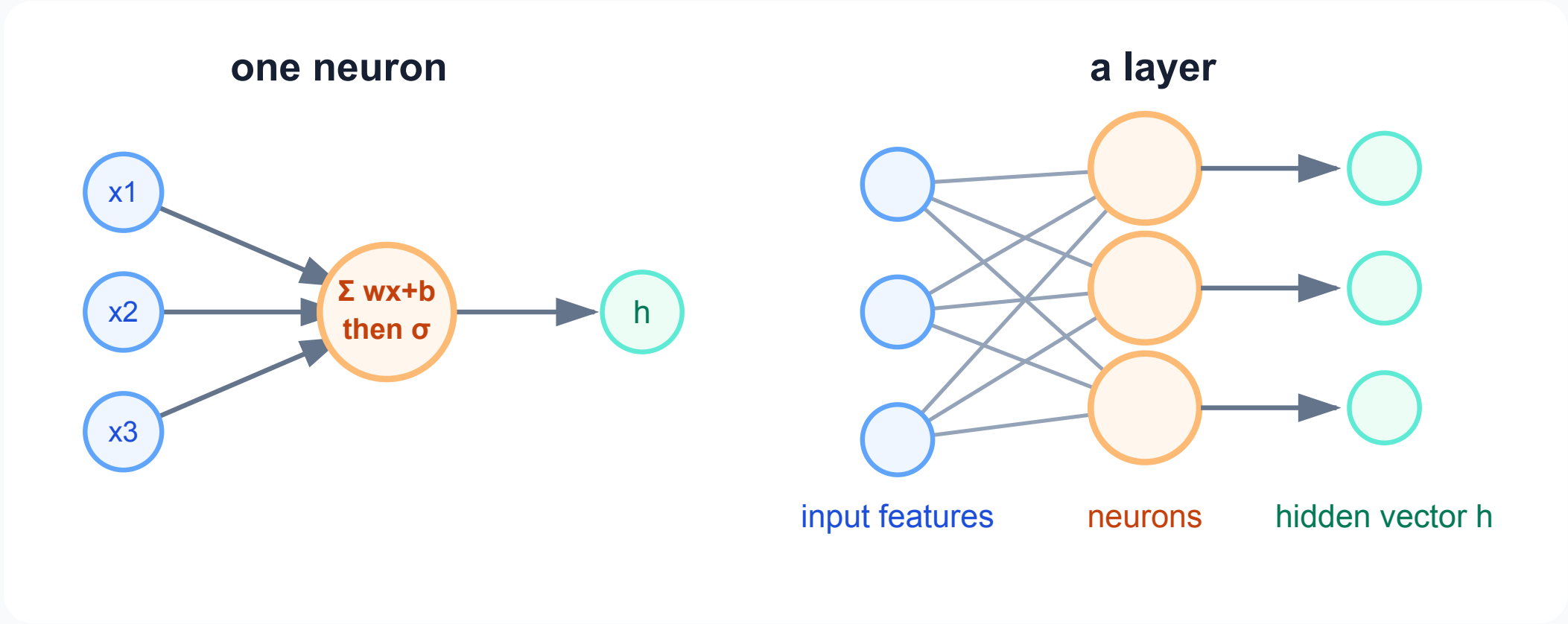
Binary logistic regression:

$$z = w^\top x + b$$
$$p = \sigma(z)$$



The difference is that deep networks stack more transformations.

From One Neuron To A Layer



Each neuron learns a different weighted pattern.

Part 3

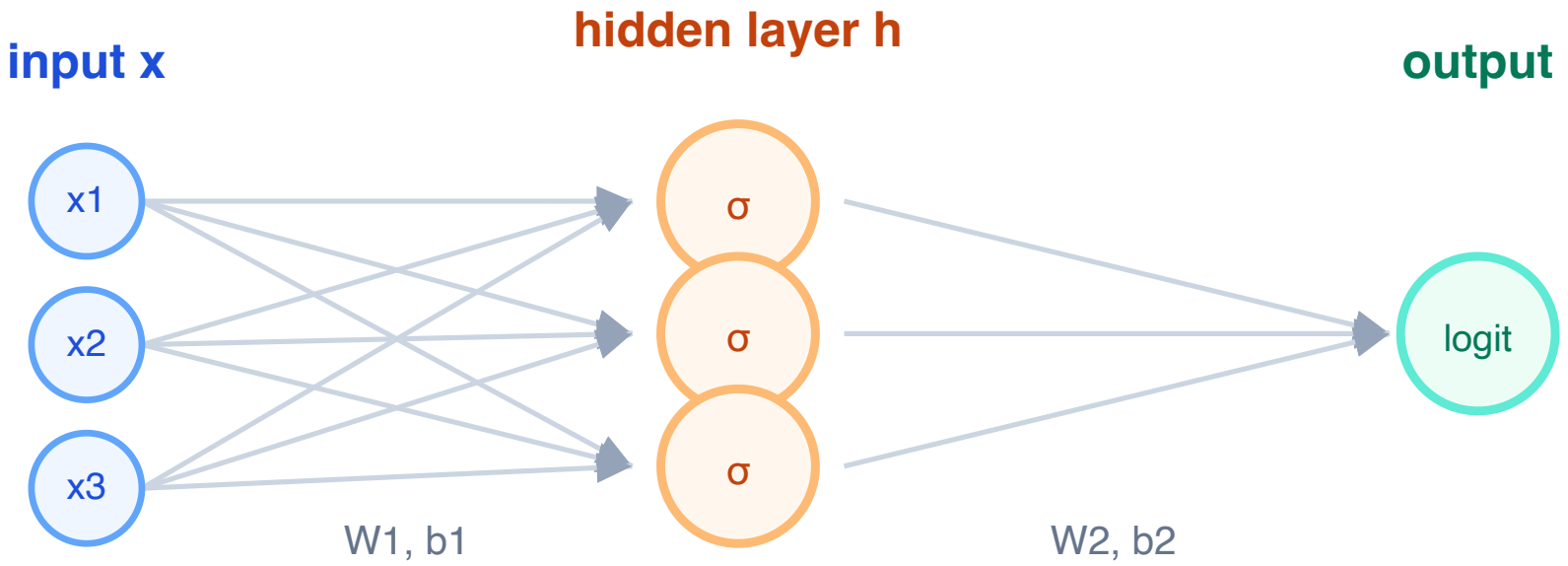
MLPs And Nonlinearity

Depth becomes useful only with nonlinear steps

Multilayer Perceptron

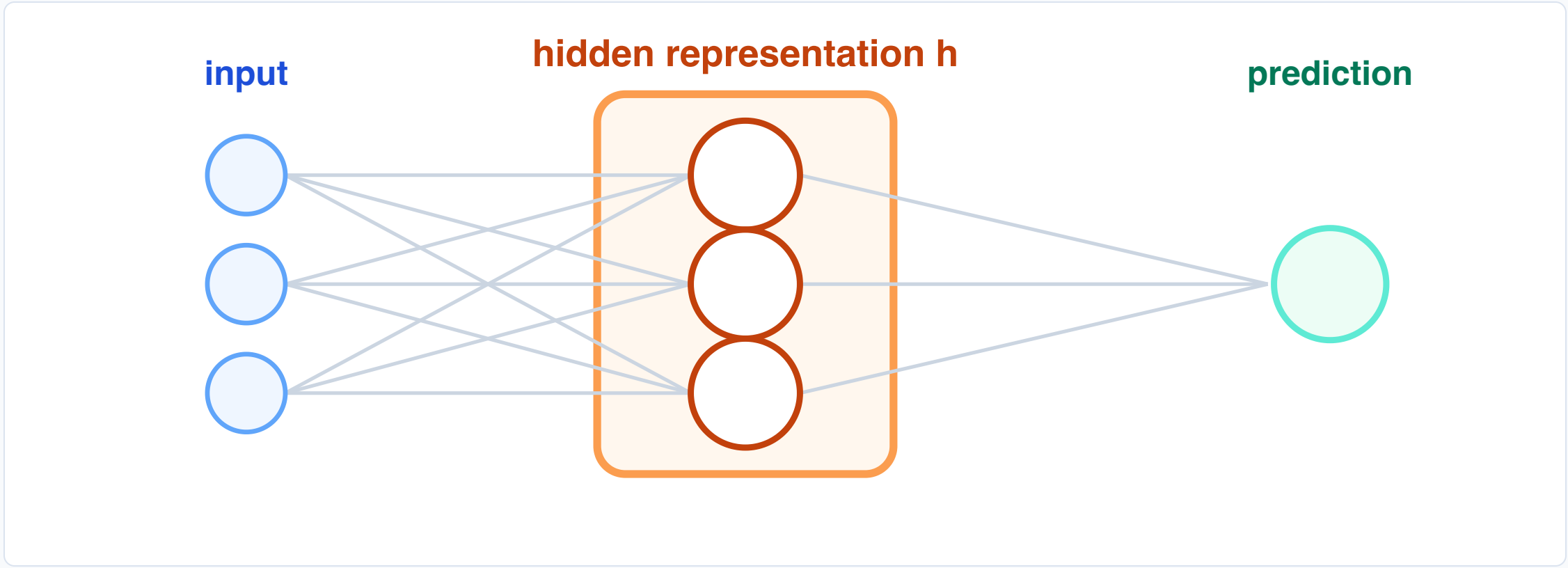
A basic MLP:

$$h = \sigma(W_1 x + b_1)$$
$$\hat{y} = W_2 h + b_2$$



Hidden Representations

The hidden layer is not the final answer. It is an intermediate description of the input.



The model learns what information should pass through this bottleneck.

Math Anchor: Linear Layers Collapse

Suppose we stack two linear layers:

$$h = W_1 x + b_1$$
$$\hat{y} = W_2 h + b_2$$

Substitute:

$$\hat{y} = W_2(W_1 x + b_1) + b_2$$
$$\hat{y} = (W_2 W_1)x + (W_2 b_1 + b_2)$$

Still just a linear model.

Key Consequence

Without nonlinearities:

linear -> **linear** -> **linear**

is equivalent to:

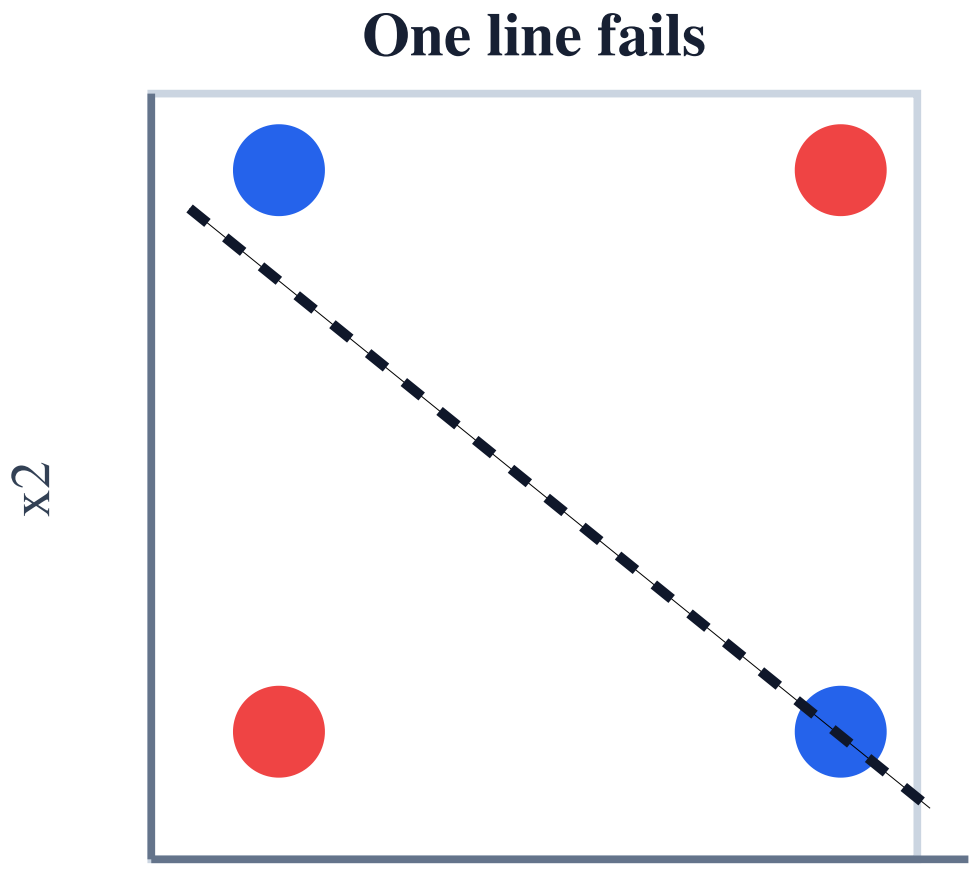
one bigger linear layer

So depth alone is not enough.

We need nonlinear activation functions.

Nonlinearity: The XOR Problem

Some patterns cannot be separated by one straight line.



class 1
$(0, 1), (1, 0)$

class 0
$(0, 0), (1, 1)$

A hidden layer can bend the boundary.

How An MLP Solves XOR

One constructive solution uses two hidden neurons.

They learn two simple tests:

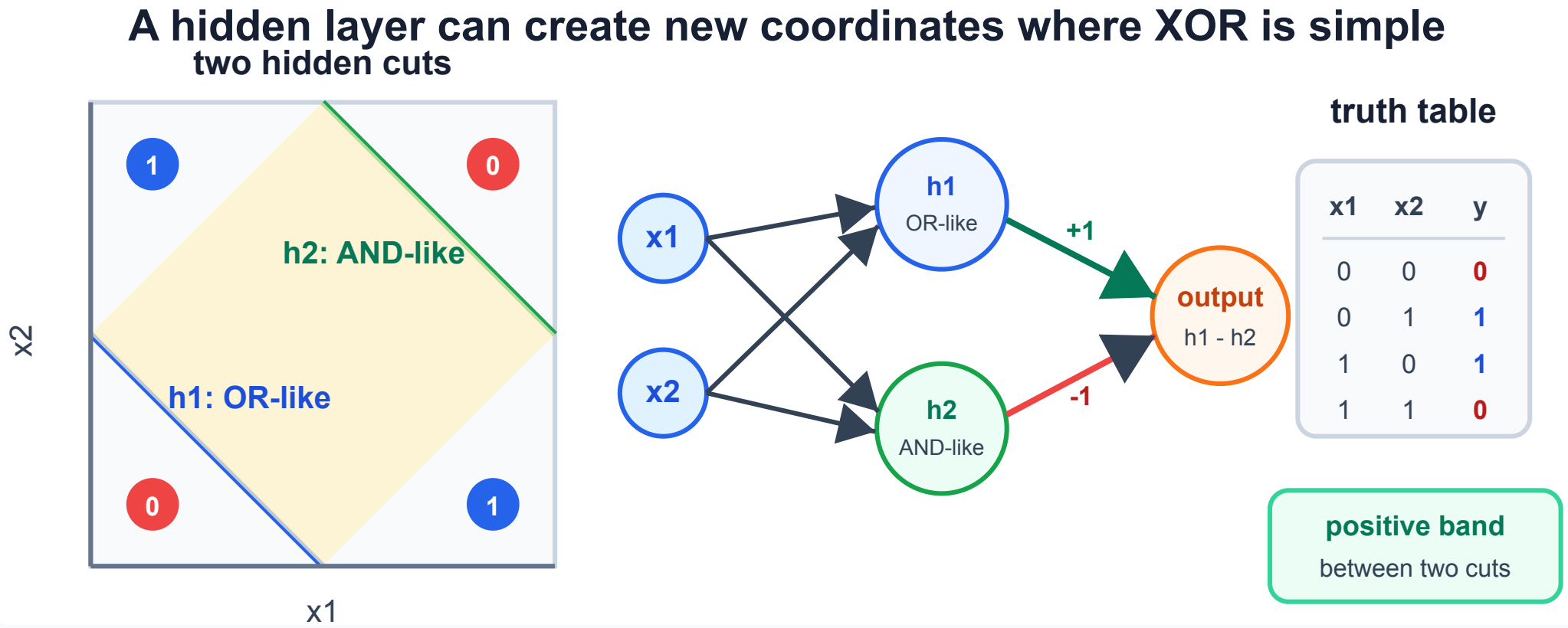
$$h_1 \approx \mathbb{1}\left[x_1 + x_2 \geq 0.5\right]$$
$$h_2 \approx \mathbb{1}\left[x_1 + x_2 \geq 1.5\right]$$

Then the output can compute:

$$y_{\text{xor}} \approx h_1 - h_2$$

So the hidden layer turns the plane into a feature space where XOR is linearly readable.

XOR: Hidden Layer Picture



Activation Functions

An activation function is applied after a layer:

$$h = \sigma(Wx + b)$$

It should usually be:

- nonlinear
- differentiable almost everywhere
- cheap to compute
- numerically stable enough for training

The activation is what prevents the network from collapsing into one linear model.

Part 4

Common Activations

What should we use inside hidden layers?

Sigmoid

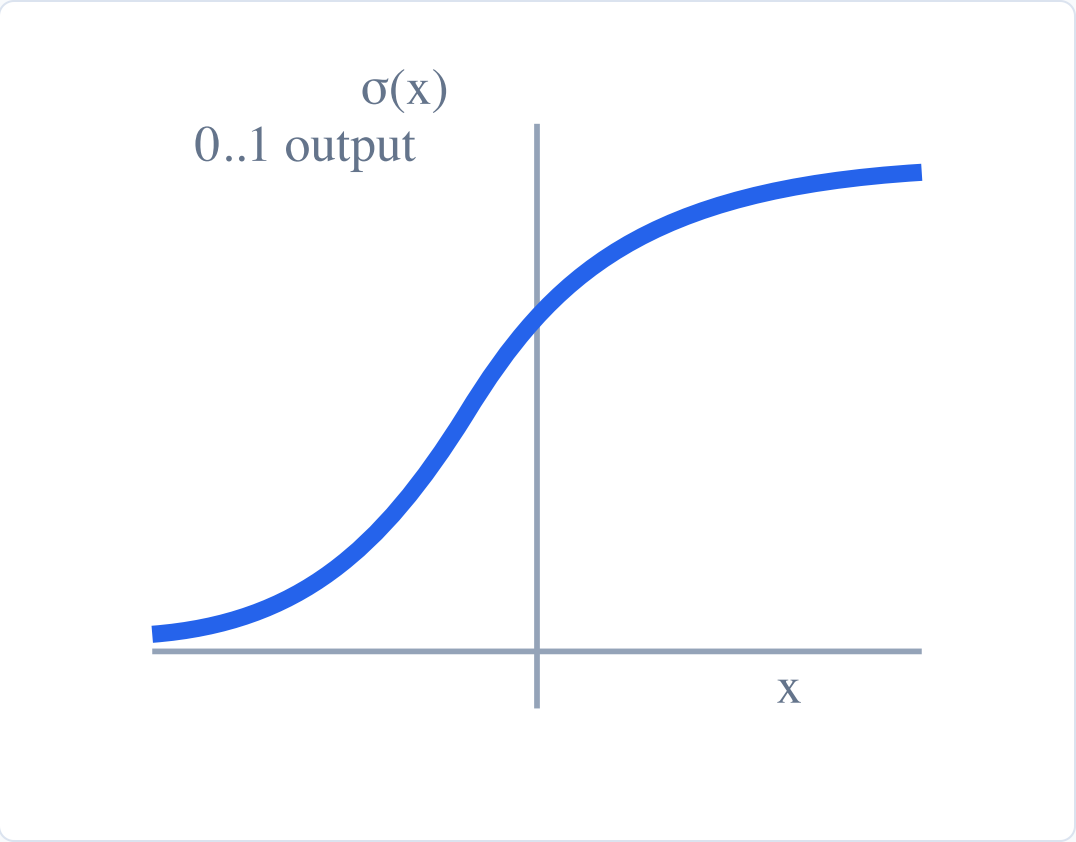
$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Properties:

- maps real numbers to $(0, 1)$
- useful for binary probabilities
- historically important

Problems:

- saturates for large positive or negative values
- gradients become very small
- output is not zero-centered



Sigmoid Saturation

When x is very positive:

```
sigmoid(x) is close to 1
gradient is close to 0
```

When x is very negative:

```
sigmoid(x) is close to 0
gradient is close to 0
```

Small gradients mean slow learning.

This is one reason sigmoid is rarely used inside modern MLP/CNN hidden layers.

Tanh

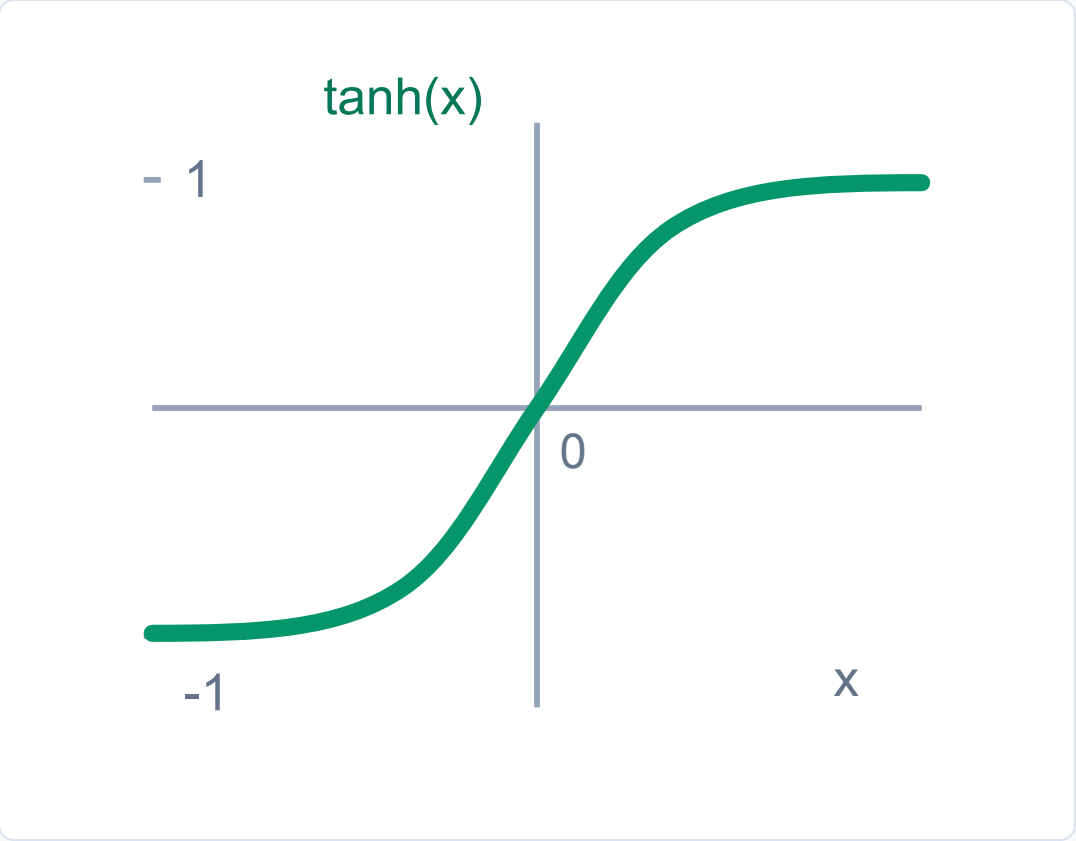
$$\tanh(x) \in (-1, 1)$$

Compared to sigmoid:

- zero-centered output
- historically popular
- still appears in some recurrent architectures

But:

- also saturates
- can still produce tiny gradients



ReLU

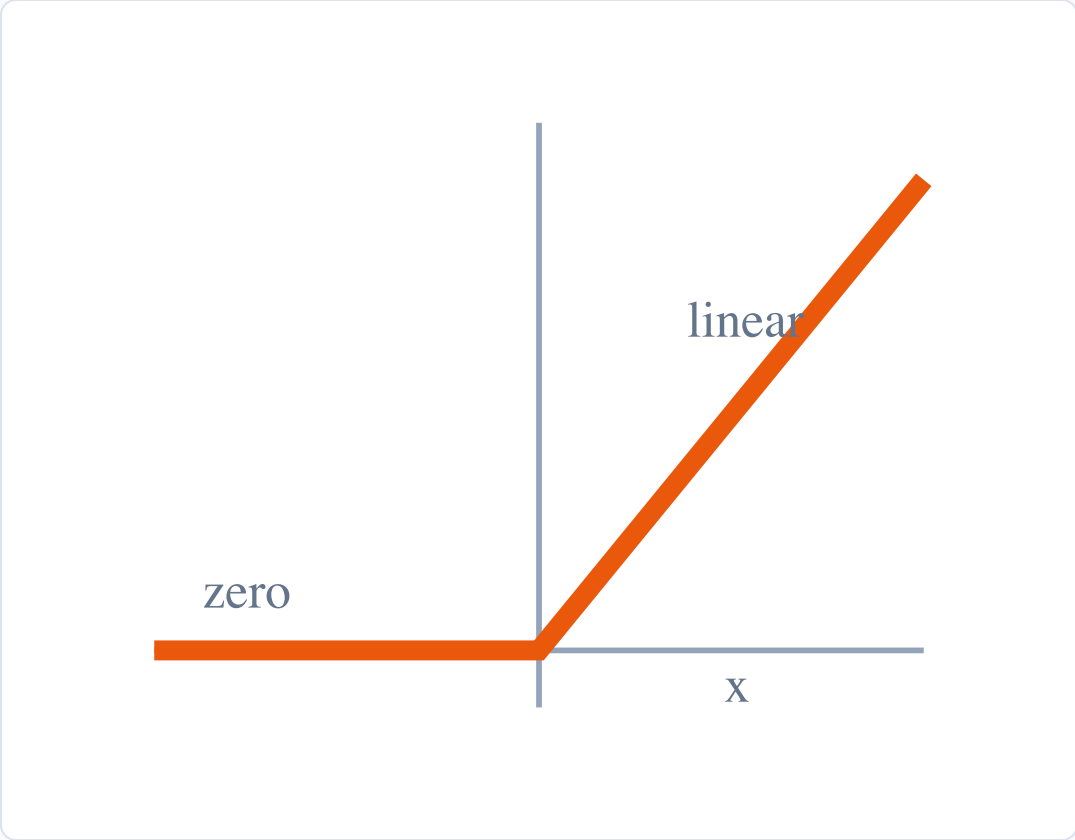
$$ReLU(x) = \max(0, x)$$

Why it became popular:

- simple
- fast
- no saturation for positive values
- works well in many deep networks

Problem:

- zero gradient when $x < 0$
- some neurons can become inactive



Dead ReLU

If a ReLU neuron always receives negative inputs:

```
output = 0
gradient = 0
```

Then it may stop learning.

This can happen with:

- bad initialization
- learning rate too high
- unlucky data/parameter dynamics

Usually ReLU still works well as a baseline.

LeakyReLU And GELU

LeakyReLU keeps a small slope for negative inputs:

$$LeakyReLU(x) = \begin{cases} x, & x > 0 \\ \alpha x, & x \leq 0 \end{cases}$$

GELU is common in transformer models.

For this course:

- MLP/CNN baseline: `ReLU`
- transformer feed-forward blocks: often `GELU`
- if ReLU behaves badly: try `LeakyReLU`

Practical Activation Advice

For hidden layers:

- start with ReLU for MLPs and CNNs
- use GELU when following transformer architecture
- try LeakyReLU if ReLU seems unstable or too sparse
- avoid sigmoid/tanh as default hidden activations

For output layers:

- binary classification often uses sigmoid logic
- multiclass classification often uses softmax logic

Important: in PyTorch, losses often expect raw logits.

Part 5

Forward Pass, Loss, Shapes

The minimum vocabulary for tomorrow's training lecture

Output Is Not Always A Probability

Neural networks often output raw scores called **logits**.

```
model output: [-1.2, 0.7, 3.1]
```

These are not probabilities yet.

For many losses, this is exactly what we want.

Example:

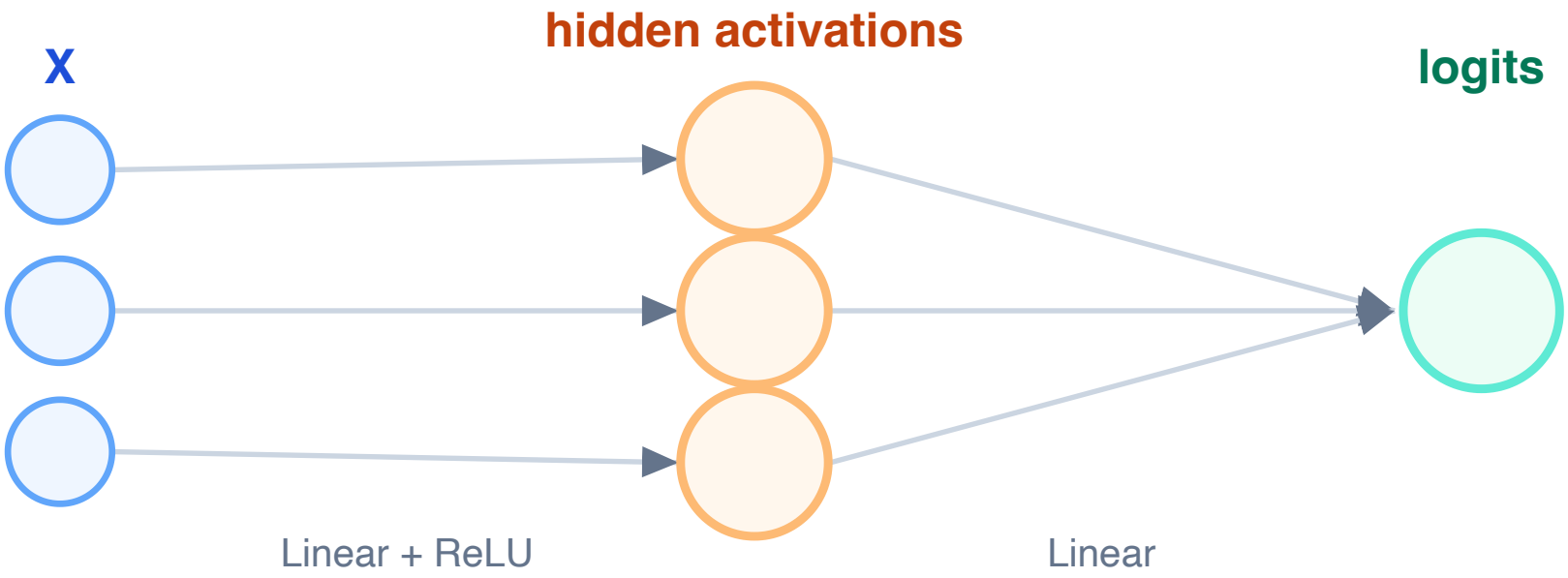
```
nn.CrossEntropyLoss()
```

expects raw logits, not softmax probabilities.

We will cover this carefully on Day 3.

Forward Pass

A forward pass means computing predictions from inputs.

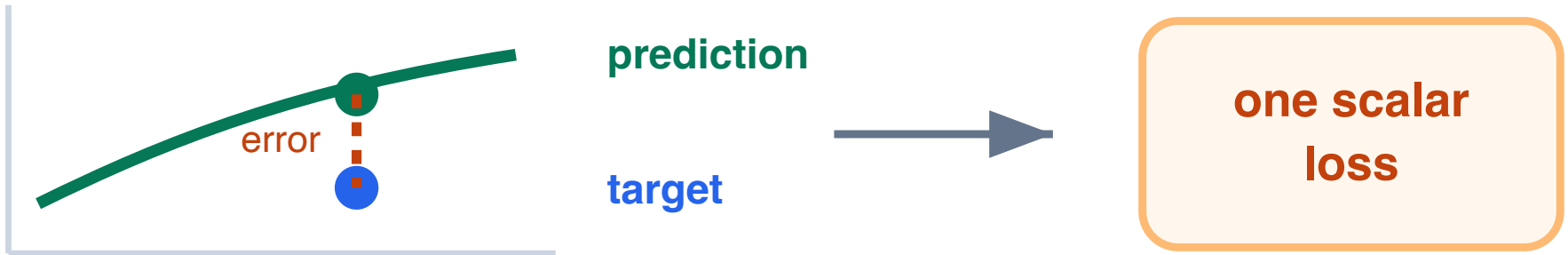


In code:

```
logits = model(X)
```

Loss Function

A loss function measures how bad the prediction is.



Examples:

- regression: mean squared error
- binary classification: binary cross-entropy
- multiclass classification: cross-entropy

Training Signal

The loss is a scalar.

That is important.

```
many predictions -> one scalar loss
```

Backpropagation starts from this scalar and computes how each parameter influenced it.

Tomorrow we will study this mechanism directly.

Shapes And Initialization Checklist

These details are not the main story, but they prevent many first-week bugs.

Shapes

```
X:      batch_size x input_dim
y:      batch_size
logits: batch_size x num_outputs
```

```
print(X.shape, y.shape, logits.shape)
```

Initialization

- do not set all weights to zero
- small random values break symmetry
- too large can explode
- too small can vanish
- PyTorch defaults are usually reasonable

Part 6

PyTorch Preview

What students should recognize in Seminar 01

A Small MLP In PyTorch

```
import torch.nn as nn

model = nn.Sequential(
    nn.Linear(2, 16),
    nn.ReLU(),
    nn.Linear(16, 1),
)
```

This means:

```
2 input features
16 hidden features
1 output logit
```

You will train this kind of model in seminar.

What Parameters Does It Have?

```
nn.Linear(2, 16)
```

Parameters:

```
weights: 16 x 2 = 32
biases:  16
total:   48
```

```
nn.Linear(16, 1)
```

Parameters:

```
weights: 1 x 16 = 16
biases:  1
total:   17
```

Mini Check

Suppose:

```
model = nn.Sequential(
    nn.Linear(4, 8),
    nn.ReLU(),
    nn.Linear(8, 3),
)
```

And:

```
X.shape == (32, 4)
```

What is the shape of `model(X)`?

Answer

```
X:                32 x 4
after Linear:     32 x 8
after ReLU:       32 x 8
after Linear:     32 x 3
```

So:

```
model(X).shape == (32, 3)
```

Interpretation:

```
32 objects, 3 output scores per object
```

Mini Check

What is wrong with this model implementation?

```
model = nn.Sequential(
    nn.Linear(10, 20),
    nn.Linear(20, 5),
    nn.Linear(5, 1),
)
```

Answer

There are many layers.

But there are no activation functions.

Composition of linear transformations is still linear.

The whole model can be rewritten as:

```
one Linear(10, 1) layer
```

Depth becomes useful only when we add nonlinear transformations between linear layers.

Seminar Bridge

In seminar, connect each coding step to today:

Seminar action	Lecture idea
create tensors	data and parameters
check shapes	valid operations
use autograd	gradients
fit a linear model	$Wx + b$
train an MLP	linear + activation + linear

Key Takeaways

- A neural network is a trainable function.
- Linear layers compute affine transformations.
- Nonlinear activations make deep models more expressive.
- ReLU is the usual baseline hidden activation.
- Tensor shapes are a core debugging tool.
- The loss gives the scalar training signal.

Tomorrow:

HW0: Derivatives Warm-Up

Release: May 18 (Today)

Deadline: May 21

Impact: low-stakes warm-up (5% of Homeworks overall grade)

Purpose:

- refresh derivatives before backpropagation
- implement losses and gradients in NumPy
- practice shape checks on a small dataset

Based on materials from girafe-ai/ml-course: https://github.com/girafe-ai/ml-course